Assignment ETL

Task 1:

task1_sales_etl.py

```
import pandas as pd
import mysql.connector
# CSV
csv_data = pd.read_csv("store_sales.csv")
# MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",
    database="sales_db"
)
mysql_data = pd.read_sql("SELECT * FROM online_sales", conn)
# JSON
json_data = pd.read_json("mobile_sales.json")
json_data.rename(columns={
    "id": "order_id",
    "item": "product",
    "price": "amount"
}, inplace=True)
json_data["date"] = pd.to_datetime(json_data["date"], dayfirst=True)
mysql_data.rename(columns={"order_date": "date"}, inplace=True)
merged_data = pd.concat([csv_data, mysql_data, json_data])
print(merged_data)
from sqlalchemy import create_engine
engine = create_engine("mysql+mysqlconnector://root:1234@localhost/sales_db")
merged_data.to_sql("sales_warehouse", engine, if_exists="replace", index=False)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task1_sales_etl.py
V:\CBA\Assignment ETL - 2\task1_sales_etl.py:14: UserWarning: pandas only supports SQLAlchemy
qlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlche
  mysql_data = pd.read_sql("SELECT * FROM online_sales", conn)
   order_id product   amount                date
0       101  Mobile  20000.0          2026-03-25
1       102  Laptop  55000.0          2026-03-25
2       103  Tablet  15000.0          2026-03-26
0       201  Mobile  21000.0          2026-03-25
1       202  Laptop  53000.0          2026-03-26
0       301  Mobile  20500.0  2026-03-25 00:00:00
1       302  Tablet  14000.0  2026-03-26 00:00:00
```

Task 2:

task2_cleaning.py

```
import pandas as pd

df = pd.read_csv("customers.csv")

# Remove duplicates

df = df.drop_duplicates()

# Handle missing emails

df = df.dropna(subset=["email"])

# Standardize phone numbers

df["phone"] = df["phone"].str.replace(r"\D", "", regex=True)

print(df)

df.to_csv("clean_customers.csv", index=False)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task2_cleaning.py
   id        name           email         phone
0   1    John Doe  john@gmail.com    9876543210
2   3    John Doe  john@gmail.com    9876543210
3   4  Ravi Kumar  ravi@gmail.com  919876543211
```

Task 3:

task3_scd.py

```
import pandas as pd
```

```
day1 = pd.read_csv("employees_day1.csv")
day2 = pd.read_csv("employees_day2.csv")
merged = pd.merge(day1, day2, on="emp_id", how="outer", suffixes=("_old", "_new"))
# Detect new records
new_records = merged[merged["name_old"].isna()]
# Detect updated records
updated = merged[merged["salary_old"] != merged["salary_new"]]
print("New Records:")
print(new_records)
print("Updated Records:")
print(updated)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task3_scd.py
New Records:
   emp_id name_old  salary_old name_new  salary_new
2       3      NaN         NaN    Sneha       45000
Updated Records:
   emp_id name_old  salary_old name_new  salary_new
0       1     Amit     50000.0     Amit       55000
2       3      NaN         NaN    Sneha       45000
```

Task 4:

task4_pyspark_logs.py

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("LogProcessing").getOrCreate()
logs = spark.read.text("logs.txt")
error_logs = logs.filter(logs.value.contains("500"))
error_logs.show()
from pyspark.sql.functions import split
```

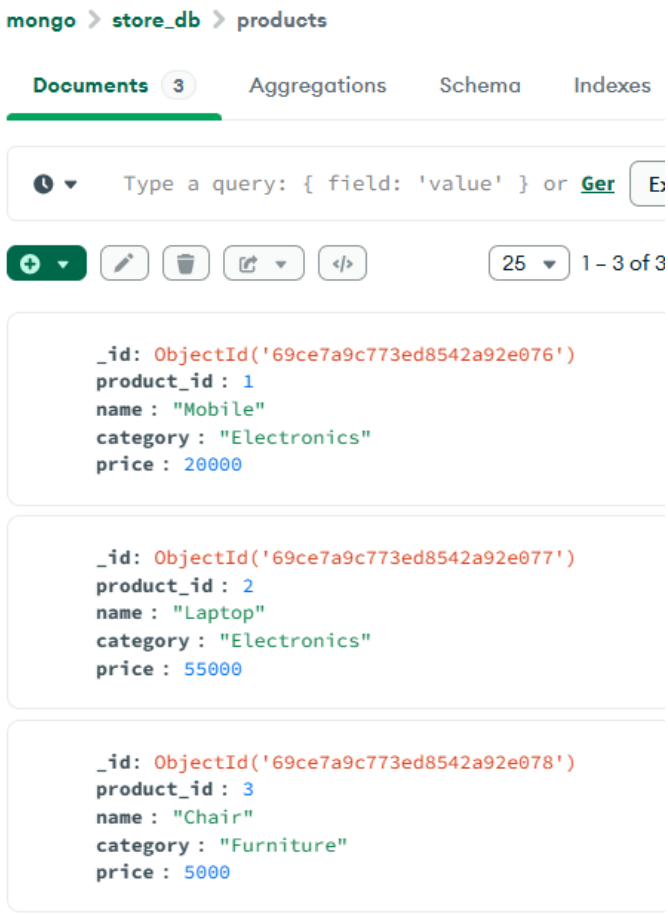
```
logs_df = logs.withColumn("date", split(logs.value, " ").getItem(0))
error_count = error_logs.withColumn("date", split(error_logs.value, " ").getItem(0)) \
                        .groupBy("date").count()
error_count.write.parquet("error_logs.parquet")
```

Task 5:

task5_mongo_migration.py

```
import pandas as pd
from pymongo import MongoClient
df = pd.read_csv("products.csv")
# Convert to nested JSON
data = df.to_dict(orient="records")
client = MongoClient("mongodb://localhost:27017/")
db = client["store_db"]
collection = db["products"]
collection.insert_many(data)
# Create index
collection.create_index("product_id")
```

output:



Task 6:

task6_streaming.py

```
import pandas as pd
df = pd.read_csv("iot_data.csv")
# Remove invalid temperature
df = df[df["temperature"] > -50]
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["minute"] = df["timestamp"].dt.floor("T")
avg_temp = df.groupby("minute")["temperature"].mean()
print(avg_temp)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task6_streaming.py
V:\CBA\Assignment ETL - 2\task6_streaming.py:9: FutureWarning: 'T'
  df["minute"] = df["timestamp"].dt.floor("T")
minute
2026-03-25 10:00:00    25.0
2026-03-25 10:01:00    26.0
2026-03-25 10:02:00    28.0
Name: temperature, dtype: float64
```

Task 7:

task7_validation.py

```
import pandas as pd

df = pd.read_csv("validation_data.csv")

# Check data types

df["age"] = pd.to_numeric(df["age"], errors="coerce")

# Reject bad records

bad_records = df[df.isnull().any(axis=1)]

# Good records

good_records = df.dropna()

print("Bad Records:")

print(bad_records)

print("Good Records:")

print(good_records)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task7_validation.py
Bad Records:
   id   name   age
1   2  Rahul   NaN
2   3  Sneha   NaN
3   4    NaN  30.0
Good Records:
   id  name   age
0   1  Amit  25.0
```

Task 8:

task8_aggregation.py

```
import pandas as pd
df = pd.read_csv("transactions.csv")
summary = df.groupby(["region", "product"])["amount"].sum()
print(summary)
```

output:

```
(venv) PS V:\CBA\Assignment ETL - 2> python .\task8_aggregation.py
region  product
East    Laptop     50000
        Mobile     22000
West    Mobile     20000
        Tablet     15000
Name: amount, dtype: int64
```

Task 9:

task9_excel_etl.py

```
import pandas as pd
from sqlalchemy import create_engine
sales = pd.read_excel("sales_excel.xlsx")
lookup = pd.read_csv("lookup.csv")
merged = pd.merge(sales, lookup, on="product")
engine = create_engine("postgresql://postgres:1234@localhost:5432/sales_db")
merged.to_sql("sales_data", engine, if_exists="replace", index=False)
```

Task 10:

task10_pyspark_join.py

```
customers = spark.read.csv("customers_large.csv", header=True)
transactions = spark.read.csv("transactions_large.csv", header=True)
from pyspark.sql.functions import broadcast
```

```
# Broadcast join
joined = transactions.join(broadcast(customers), "cust_id")
joined.show()
```